

WeatherStation

ESP32-S3 Firmware Documentation

Version	v0.2.0
Description	Increment 1 - FreeRTOS, WiFi, deep-sleep, RTC/NTP
MCU	ESP32-S3-DevKitC-1
Framework	Arduino (PlatformIO)
Generated	15 April 2026
Author	Carlo Falco
Repository	https://github.com/CarloFalco/WeatherStation

Hardware Overview

Sensor	Interface	Measurement
BME280	I2C 0x76	Temperature / Humidity / Pressure
AS5600	I2C 0x36	Wind direction (magnetic encoder)
Rain gauge	GPIO19 reed switch	Rainfall accumulation
Anemometer	GPIO20 reed switch	Wind speed
PMSA003I	UART RX=17 TX=18	PM1.0, PM2.5, PM10
CCS811	I2C 0x5A	eCO2, TVOC
MICS6814	ADC GPIO5/6/7	CO, NO2, NH3
INA3221	I2C 0x40	Voltage, Current (3 channels)

Table of Contents

- **1. Architecture Overview**
 - 1.1 Boot Sequence
 - 1.2 Module Map
- **2. Hardware Reference**
 - 2.1 GPIO Pinout
 - 2.2 I2C Addresses
- **3. Configuration Reference (config.ini)**
 - 3.1 [wifi]
 - 3.2 [mqtt]
 - 3.3 [ota]
 - 3.4 [sampling]
- **4. API Reference**
 - include\config.h
 - include\rtc_state.h
 - include\task_events.h
 - include\version.h
 - src\main.cpp
 - src\config\AppConfig.h
 - src\connectivity\WiFiManager.h
 - src\power\PowerManager.h
 - src\provisioning\Provisioning.h
 - src\rtc\TimeManager.h
- **5. Versioning & Release Process**
 - 5.1 Roadmap
 - 5.2 Release Checklist

1. Architecture Overview

1.1 Boot Sequence

1	Serial + LittleFS init	400 ms USB CDC settle delay; format-on-fail
2	Provisioning check	Reset pin held >= 3 s OR /config.ini missing → see config section
3	Captive-portal wizard	Soft-AP + DNS + HTTP; blocks until restart
4	AppConfig::load()	Parses /config.ini; falls back to provisioning on error (AppConfig API)
5	FreeRTOS tasks	Increment 1: WiFiTask, SleepTask (placeholder)

1.2 Module Map

Path	Class	Responsibility
src/config/	AppConfig	INI parser / serialiser singleton
src/provisioning/	Provisioning	Soft-AP, DNS, HTTP captive portal
src/power/	PowerManager	MOSFET rails, deep-sleep (Increment 1)
src/connectivity/	WiFiManager	Wi-Fi connect / reconnect (Increment 1)
src/sensors/	Sensor*	Per-sensor driver classes (Increment 2)
src/ota/	OtaManager	GitHub Releases OTA (Increment 4)

2. Hardware Reference

2.1 GPIO Pinout

Signal	GPIO	Direction	Notes
I2C SDA	GPIO8	Bidirect	BME280, AS5600, CCS811, INA3221
I2C SCL	GPIO9	Output	Shared I2C clock
Rain gauge	GPIO19	Input	Reed switch, active LOW, PULLUP
Anemometer	GPIO20	Input	Reed switch, active LOW, PULLUP
PM25 UART RX	GPIO17	Input	Sensor TX to ESP RX
PM25 UART TX	GPIO18	Output	ESP TX to Sensor RX
CCS811 WAKE	GPIO41	Output	Drive LOW before I2C access
MICS6814 CO	GPIO5	ADC In	ADC1 Channel 4
MICS6814 NO2	GPIO6	ADC In	ADC1 Channel 5
MICS6814 NH3	GPIO7	ADC In	ADC1 Channel 6
5V MOSFET enable	GPIO11	Output	Active LOW (P-ch MOSFET)
3.3V MOSFET enable	GPIO4	Output	Active LOW (P-ch MOSFET)
Reset / config pin	GPIO0	Input	Hold LOW >= 3 s for factory reset
Built-in LED	GPIO97	Output	Active HIGH

2.2 I2C Addresses

Device	Address	Library
BME280	0x76	Adafruit_BME280_Library
AS5600	0x36	AS5600
CCS811	0x5A	Adafruit_CCS811
INA3221	0x40	INA3221_WE

3. Configuration Reference (config.ini)

The runtime configuration is stored at `/config.ini` on LittleFS and is written by the [Provisioning wizard](#). All parameters are exposed through the [AppConfig singleton](#).

[wifi]

Key	Type	Default	Description
<code>ssid</code>	string		Wi-Fi network SSID (required)
<code>password</code>	string		Password; empty = open network

[mqtt]

Key	Type	Default	Description
<code>host</code>	string		Broker hostname or IP (required)
<code>port</code>	uint16	1883	TCP port
<code>user</code>	string		Username (optional)
<code>password</code>	string		Password (optional)
<code>client_id</code>	string	weather-station-01	Unique MQTT client ID
<code>base_topic</code>	string	weather/station01	Root topic prefix

[ota]

Key	Type	Default	Description
<code>github_repo</code>	string	CarloFalco/WeatherStation	GitHub repo used for OTA firmware download
<code>github_token</code>	string		Personal Access Token (leave empty for public repos)

[sampling]

Key	Type	Default	Description
<code>bme280_s</code>	uint32	60	BME280 read interval (s, min 5)
<code>rain_s</code>	uint32	60	Rain gauge window (s, min 10)
<code>wind_s</code>	uint32	10	Anemometer sample (s, min 5)
<code>pm25_s</code>	uint32	120	PM2.5 interval (s, min 10)
<code>ccs811_s</code>	uint32	120	CCS811 interval (s, min 10)
<code>mics_s</code>	uint32	120	MICS6814 interval (s, min 10)
<code>ina_s</code>	uint32	60	INA3221 interval (s, min 5)
<code>publish_s</code>	uint32	300	MQTT publish interval (s)

<code>sleep_s</code>	<code>uint32</code>	300	Deep-sleep duration (s); 0 = disabled
----------------------	---------------------	-----	---------------------------------------

4. API Reference

Extracted automatically from `/** ... */` Doxygen-style comments in the source files.

`include/config.h`

Compile-time hardware pin and peripheral address definitions. This file is the single source of truth for all GPIO assignments and I²C/UART addresses used by the ESP32-S3 Weather Station firmware.

Note: I²C bus pins (PIN_I2C_SDA / PIN_I2C_SCL) are set to GPIO8 / GPIO9 by default, which matches the ESP32-S3-DevKitC-1 reference design. Modify here if your board uses different pins.

- **@brief I²C SDA pin.**
- **@brief I²C SCL pin.**
- **@brief BME280 I²C address (SDO pulled to GND → 0x76).**
- **@brief GPIO for the rain-gauge reed switch (interrupt-driven, active LOW).**
- **@brief GPIO for the anemometer reed switch (interrupt-driven, active LOW).**
- **@brief AS5600 I²C address (fixed, 0x36).**
- **@brief UART RX pin connected to sensor TX.**
- **@brief UART TX pin connected to sensor RX.**
- **@brief Hardware UART port used for the PM25 sensor.**
- **@brief CCS811 I²C address (ADDR pin pulled to GND → 0x5A).**
- **@brief Active-LOW WAKE pin of the CCS811 (must be driven LOW before I²C).**
- **@brief ADC1 channel for the CO channel of the MICS6814. GPIO5.**
- **@brief ADC1 channel for the NO2 channel of the MICS6814. GPIO6.**
- **@brief ADC1 channel for the NH3 channel of the MICS6814. GPIO7.**
- **@brief INA3221 I²C address (A0 pulled to GND → 0x40).**
- **GPIO controlling the P-channel MOSFET that switches 5 V rail.**
- **GPIO controlling the P-channel MOSFET that switches 3.3 V rail.**
- **@brief On-board LED (active HIGH).**
- **@brief GPIO used to force provisioning mode at boot.**

- @brief Debounce window (ms) applied to reed-switch interrupts.
- @brief Time (ms) the BOOT button must be held to trigger a config reset.

`include/rtc_state.h`

*Variables stored in RTC slow memory – retained across deep-sleep cycles. Declared `extern` here and ****defined once**** in `main.cpp`. Any translation unit that includes this header can read / write these values.*

- @brief Number of wake cycles since the last power-on / hard reset.
- @brief UNIX epoch (seconds since 1970-01-01 UTC) saved just before
- @brief Deep-sleep duration used in the previous cycle (seconds).
- @brief True once NTP has synchronised at least once since hard reset.

`include/task_events.h`

- @brief System-wide FreeRTOS event group created in setup().
- @brief WiFi STA is connected and has a valid IP address.
- @brief WiFi connection failed after MAX_RETRIES attempts.
- @brief NTP time synchronisation completed successfully.
- @brief MQTT broker connected (Increment 3).
- @brief All sensor readings collected (Increment 2).
- @brief MQTT JSON payload published (Increment 3).

`include/version.h`

- @brief Major version – incremented on breaking changes.
- @brief Minor version – incremented on backward-compatible new features.
- @brief Patch version – incremented on backward-compatible bug fixes.
- @brief Full version string in the format "MAJOR.MINOR.PATCH".
- @brief Full version string prefixed with 'v', matching GitHub tag format.
- @brief Human-readable description of this release.

src/main.cpp

Application entry point – Increment 1: FreeRTOS, WiFi, deep-sleep, RTC/NTP. Boot sequence: 1. Initialise Serial and LittleFS. 2. Increment `g_bootCount` (RTC-persistent wake-cycle counter). 3. Check provisioning trigger (reset pin held ≥ 3 s, or missing config). 4. Load runtime configuration from ``/config.ini``. 5. Initialise PowerManager – configure MOSFET GPIO pins, print wakeup cause. 6. Initialise TimeManager – restore approximate epoch from RTC memory. 7. Create global FreeRTOS EventGroup (`g_sysEvents`). 8. Start WiFiManager task (`TaskWifi`). 9. Start sleep-orchestration task (`TaskSleep`). 10. `TaskSleep`: connect WiFi → (optionally) sync NTP → sleep.

■ **@brief FreeRTOS stack depth (words) for the sleep-orchestration task.**

■ **@brief FreeRTOS priority for TaskSleep.**

■ **FreeRTOS task that orchestrates each wake cycle. Per-cycle sequence: 1. Enable sensor rails via PowerManager. 2. Wait up to 30 s for WiFi (EVT_WIFI_CONNECTED or EVT_WIFI_FAILED). 3. If connected and NTP re-sync is due, call TimeManager::syncNtp(). 4. (Increment 2) Read all sensors. 5. (Increment 3) Publish JSON payload via MQTT. 6. Disconnect WiFi, save epoch to RTC, enter deep sleep.**

src/config/AppConfig.h

Runtime configuration loaded from an INI file stored in LittleFS. `AppConfig` is a singleton that reads a simple ``key = value`` INI file (sections delimited by ``[section]`` headers) and exposes every user-configurable parameter to the rest of the firmware. The configuration file is located at ``/config.ini`` on the LittleFS partition and is written by the provisioning captive-portal during the first-time setup (or after a factory reset).

■ **Returns the singleton AppConfig instance.**

■ **Loads configuration from an INI file on LittleFS. Parses every ``key = value`` line and populates the public member fields. Lines starting with ``;`` or ``#`` are treated as comments and ignored.**

Parameter	Description
<code>path</code>	LittleFS absolute path (e.g. <code>`/config.ini`</code>).

■ **Persists the current in-memory configuration to LittleFS. Overwrites the file at ``path`` with the serialised INI content.**

Parameter	Description
<code>path</code>	LittleFS absolute path (e.g. <code>`/config.ini`</code>).

■ **@brief Returns ``true`` when the minimum required fields are populated.**

■ **@brief Prints all configuration values to Serial (DEBUG only).**

■ **Applies a single parsed key/value pair from a given INI section.**

Parameter	Description
<code>section</code>	Current INI section name (without brackets).
<code>key</code>	Key string (trimmed, lower-case).

■ Trims leading and trailing whitespace from a String in place.

`src/connectivity/WiFiManager.h`

Manages the WiFi STA lifecycle in a dedicated FreeRTOS task.

■ @brief FreeRTOS stack depth (words) allocated for TaskWifi.

■ @brief FreeRTOS priority for TaskWifi.

■ @brief Returns the singleton instance.

■ Store credentials and spawn the WiFi management task.

Parameter	Description
<code>ssid</code>	Target network SSID.

■ Block the calling task until WiFi connects or the timeout expires. Waits for ``EVT_WIFI_CONNECTED`` **or** ``EVT_WIFI_FAILED`` in ``g_sysEvents`` — whichever comes first.

Parameter	Description
<code>timeoutMs</code>	Maximum wait in milliseconds.

Returns: ``true`` if ``EVT_WIFI_CONNECTED`` was set within the timeout.

■ @brief ``true`` while the STA interface has a valid IP address.

■ @brief Disconnect STA and power off the WiFi radio.

■ @brief FreeRTOS task entry point.

■ @brief Arduino WiFiEvent callback registered in taskFunction().

■ @brief Maximum consecutive connection failures per wake cycle.

■ @brief Delay between reconnection attempts (ms).

`src/power/PowerManager.h`

Controls sensor power rails and coordinates deep-sleep entry. Responsibilities: - Drive P-channel MOSFET gates for the **5 V** (`PIN_PWR_5V`) and **3.3 V** (`PIN_PWR_3V3`) sensor supply rails. - Configure ESP32-S3 wakeup sources before calling ``esp_deep_sleep_start()``: - Timer wakeup (periodic measurement cycle). - EXT1 GPIO wakeup on reed-switch inputs (rain gauge / anemometer) so that pulse events are never missed while the MCU is sleeping. - Report the wakeup cause on the next boot.

Note: Both MOSFET gates are driven HIGH (off) by `init()` so the sensor rails are **disabled by default** at every boot. The sleep orchestration task (`vTaskSleep`) enables them explicitly before reading sensors.

■ @brief Returns the singleton instance.

■ @brief Configures MOSFET GPIO pins and disables both sensor rails.

- Enable or disable the 5 V sensor power rail.
- Enable or disable the 3.3 V sensor power rail.
- Disable all rails, configure wakeup sources, enter deep sleep. Wakeup sources configured: 1. ****Timer**** – wakes after @p seconds. 2. ****EXT1**** – wakes when PIN_RAIN_GAUGE or PIN_ANEMOMETER goes LOW (reed switch closes). Requires external pull-ups on those lines.

Parameter	Description
seconds	Deep-sleep duration in seconds.

- @brief Returns the hardware wakeup cause of the current boot.
- @brief `true` when the current boot was triggered by the periodic timer.
- @brief `true` when the current boot was triggered by a reed-switch GPIO.
- @brief Logs a human-readable wakeup reason to Serial.

src/provisioning/Provisioning.h

First-time setup wizard via Wi-Fi Access Point and captive portal. When the firmware detects that no valid configuration exists (or the user holds the reset pin), the Provisioning module: 1. Starts the ESP32 in ****Soft-AP**** mode (SSID: "WeatherStation-Setup"). 2. Launches a ****DNS server**** that resolves every hostname to the ESP's own IP address, triggering the OS captive-portal detection on phones/tablets. 3. Serves a ****single-page HTML form**** (loaded from LittleFS `/index.html`) on port 80 where the user enters: - Wi-Fi credentials - MQTT broker settings - GitHub repository for OTA updates - Station identity / topic prefix 4. On form submission (POST /save), the values are validated, written to `/config.ini` via AppConfig::save(), and the device ****restarts****.

- @brief SSID broadcast by the provisioning Access Point.
- @brief Password for the provisioning AP (empty = open network).
- @brief IP address of the provisioning AP (default gateway seen by clients).
- @brief DNS port – must be 53.
- @brief HTTP port for the configuration web server.
- @brief Constructs a Provisioning instance (does not start AP yet).
- @brief Initialises the AP, DNS server, and HTTP server.
- @brief Processes pending DNS queries and HTTP requests.
- @brief Returns true once the user has submitted valid credentials.

- **@brief** Serves the main configuration HTML page.
- **@brief** Handles ``POST /save`` – validates and persists the form data.
- **@brief** Returns a minimal inline HTML error page.
- **Sends a styled inline HTML response (used for success/error pages).**

Parameter	Description
<code>title</code>	Page title.
<code>body</code>	HTML body content.

- **Extracts and URL-decodes a POST parameter by name.**

Parameter	Description
<code>name</code>	Parameter key as sent by the HTML form.

`src/rtc/TimeManager.h`

Manages system time via NTP and persists the epoch across deep-sleep cycles.

- **@brief** POSIX timezone string – Central European Time (CET/CEST).
- **@brief** Primary NTP server hostname.
- **@brief** Secondary NTP server hostname.
- **@brief** Default NTP synchronisation timeout (ms).
- **@brief** Number of wake cycles between NTP re-synchronisations.
- **@brief** Returns the singleton instance.
- **@brief** Restores system time from RTC slow memory at boot.
- **Synchronise system time with NTP. Calls ``configTime()`` and waits up to `@p timeoutMs` for a valid timestamp to become available. On success, sets ``EVT_NTP_SYNCED`` in ``g_sysEvents`` and saves the flag to RTC memory.**

Parameter	Description
<code>timeoutMs</code>	Maximum wait in milliseconds (default 15 000).

- **@brief** Returns ``true`` when the system time is considered valid.
- **@brief** Returns the current UNIX epoch (UTC).
- **@brief** Returns an ISO 8601 UTC timestamp string.
- **@brief** Saves the current epoch into ``g_lastEpoch`` (RTC slow memory).

- **@brief Prints the current local time to Serial (debug helper).**
- **@brief Returns ``true`` when an NTP re-sync should be performed this boot.**

5. Versioning & Release Process

This project follows [Semantic Versioning](#). Version constants are defined in [include/version.h](#) and consumed at build time by `post_build.py` and `generate_docs.py`.

5.1 Roadmap

Tag	Milestone	Status
v0.1.0	Increment 0 – Scaffold & provisioning	Released
v0.2.0	Increment 1 – FreeRTOS, deep-sleep, Wi-Fi, RTC	Planned
v0.3.0	Increment 2 – Sensor drivers	Planned
v0.4.0	Increment 3 – MQTT JSON publishing	Planned
v0.5.0	Increment 4 – OTA via GitHub Releases API	Planned

5.2 Release Checklist

- Update [include/version.h](#): `FW_VERSION_MAJOR / MINOR / PATCH`
- Add entry in `CHANGELOG.md` under the new version tag
- Run `pio run` – `post_build.py` writes `version.txt`, versioned `.bin` and PDF
- Commit: `git add . && git commit -m "chore: bump to vX.Y.Z"`
- Tag: `git tag -a vX.Y.Z -m "Release vX.Y.Z"`
- Push: `git push origin main && git push origin vX.Y.Z`
- GitHub Actions builds firmware + PDF and creates the [GitHub Release](#) automatically